

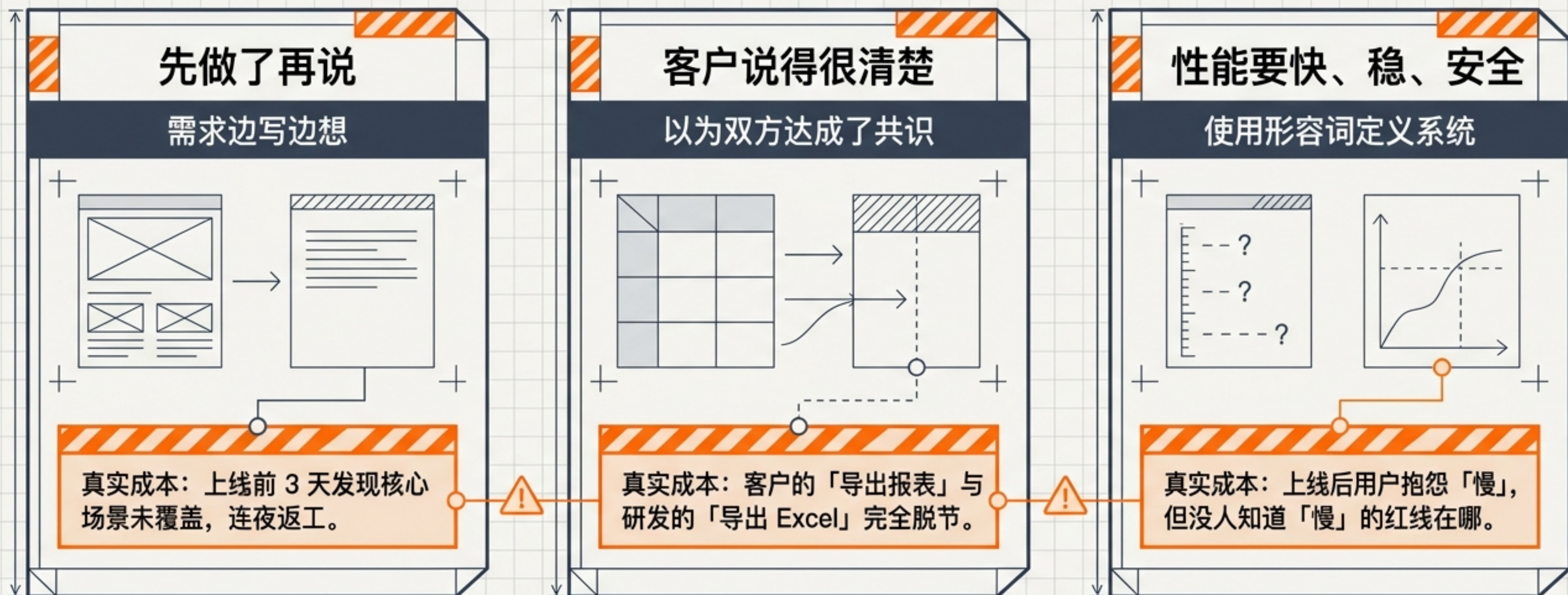
需求工程蓝图：从一句话到可执行规范

构建高投资回报率（ROI）的最小需求骨架

时长：30 分钟 | 受众：产品与研发

抛弃凭感觉做需求——用三个结构化工具将「混沌」转化为「清晰」。

软件开发的共同灾难：源于「根基不稳」



共同根因：不是开发能力不足，而是从一开始就没把「做什么、为谁做、做到什么程度」对齐。

解剖「一句话需求」的隐藏灾难

业务需求 (Business)

组织为什么要做？

(目标：把人力运营成本降低 50%)

用户需求 (User)

用户做什么，想得到什么？

(目标：候选人扫码报名，审核老师批量审批)

我们要做一个学员训练营报名系统。

软件需求 (Software)

系统要实现什么功能、达到什么质量？

(目标：提供 POST /enrollments 接口)



核心法则：直接跳过业务/用户层去做软件需求 = 大概率做错事。

需求分层透视矩阵：你在哪一层对话？

层次	回答的核心问题	谁来提/写	最终产物
 业务需求 (Business)	为什么要做？ (必须包含背景 + 可度量目标)	项目发起人 → 产品负责人	项目愿景与范围文档
 用户需求 (User)	用户做什么、想得到什么价值？	产品 / 需求分析师	用例文档 / 用户故事
 软件需求 (Software)	系统要实现什么功能、达到什么质量？	研发 / 架构师	SRS（软件需求规格说明）

Golden Rule

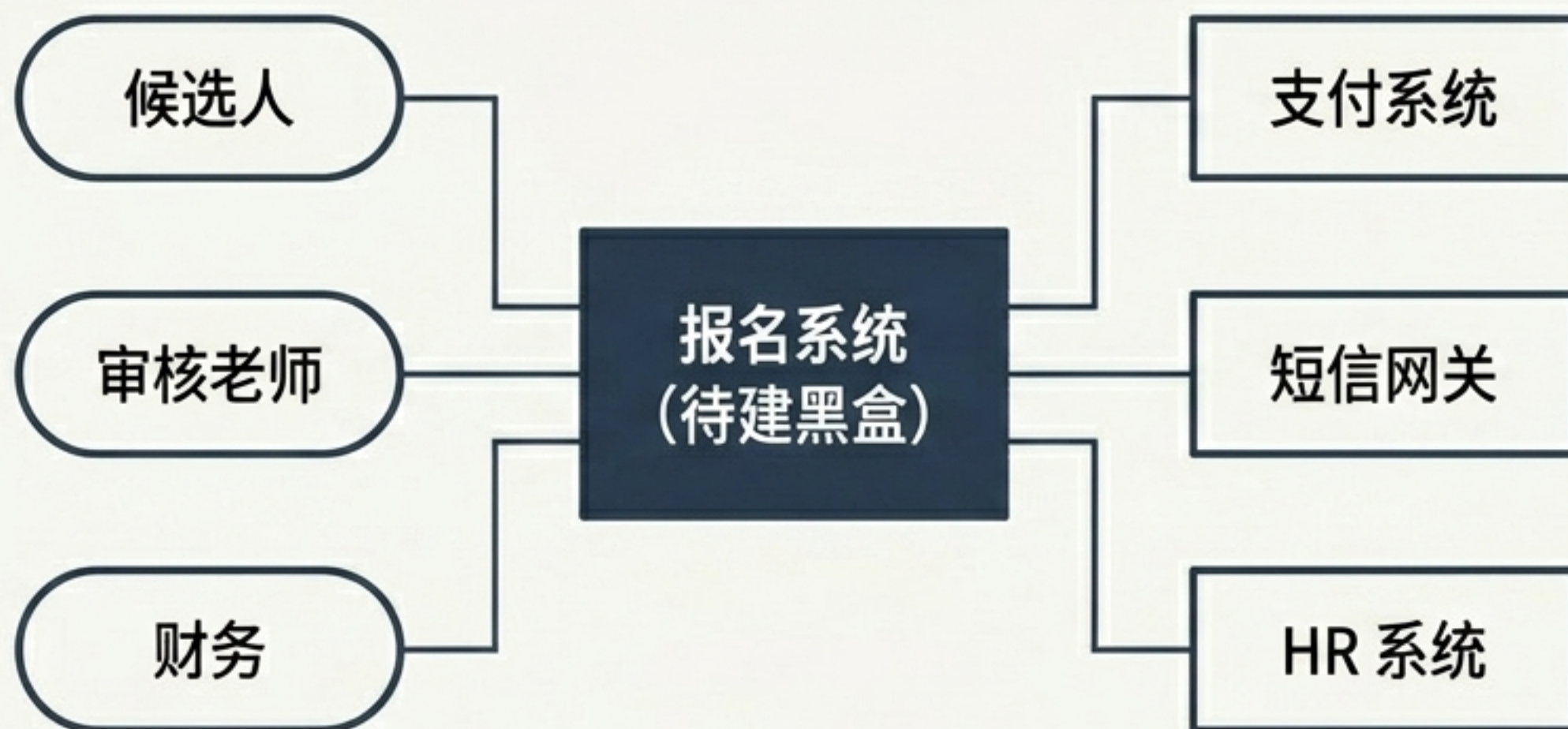
业务目标必须可度量。没有数字，就没有验收标准。例如：将平均反馈时间从 5 天降到 1 天。

工具一：上下文图 (Context Diagram)

先定义边界，再分析功能

4 Elements

1. **待建系统：**
中央黑盒（不关心内部实现）
2. **用户/角色：**直接交互的人
（圆角胶囊）
3. **外部系统：**上下游、
第三方系统（矩形）
4. **连接器：**交互关系，不关心
具体数据内容（实线）



洞察：这张图立刻暴露出，至少有 6 条接口要在边界上说清楚。

上下文图：三个最致命的画法误区

Error 1: 硬件拟人 (Hardware as Actors)



Error 2: 俄罗斯套娃 (Nested 'Inception')



Error 3: 内部暴露 (Leaking Internals)



误区：子系统单独画一张上下文图，绝不能多层嵌套。

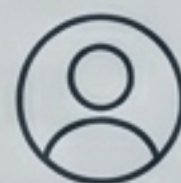
误区：上下文图只画外部连接，内部模块设计推迟到架构阶段。

黄金准则：上下文图只问 3 件事：
用户是谁？依赖谁？被谁依赖？

工具二：用例（Use Case）

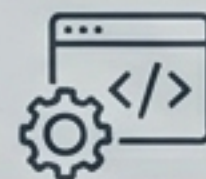
— 站在用户角度的外部特征

定义：系统为用户做了什么，而不是用户自己做了什么。必须包含可观测、有价值的结果。



用例（用户视角 - 面向业务/测试）

- 提交报名
- 缴纳学费
- 查询进度



功能（系统视角 - 面向开发）

- 表单校验 / 数据写库
- 调支付接口 / 收回调 / 写订单
- 查询服务 + 缓存

最易翻车点：把后端功能逻辑菜单当作用例图来画，是需求分析中最常见的致命错误。

用例粒度校准：「刚刚好」的艺术



判断口诀：

- 能不能写出【前置条件 / 后置条件 / 基本事件流】？
- 能写出来就是合理粒度，写不出来就是粒度不对。

用例标准描述骨架：6 大核心字段

CASE: UC-ENROLL-01

1. 名称 (Name): 提交报名

必须是动宾格式!

提交报名 ✓ / 报名 ✗

2. 简述 (Summary): 谁 / 什么场景 / 做什么 / 目标。

3. 前置条件 (Pre-condition): 训练营处于开放状态且邮箱未提交过。

必须是系统可检测的状态,
绝不是「电脑已开机」

4. 后置条件 (Post-condition): 成功则记录入库状态 pending; 失败则系统状态不变。

必须包含异常失败时的系统状态

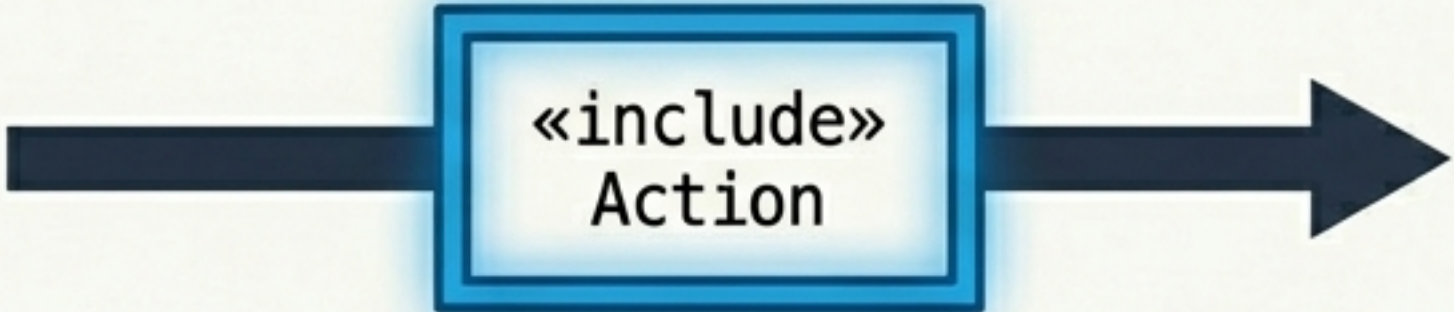
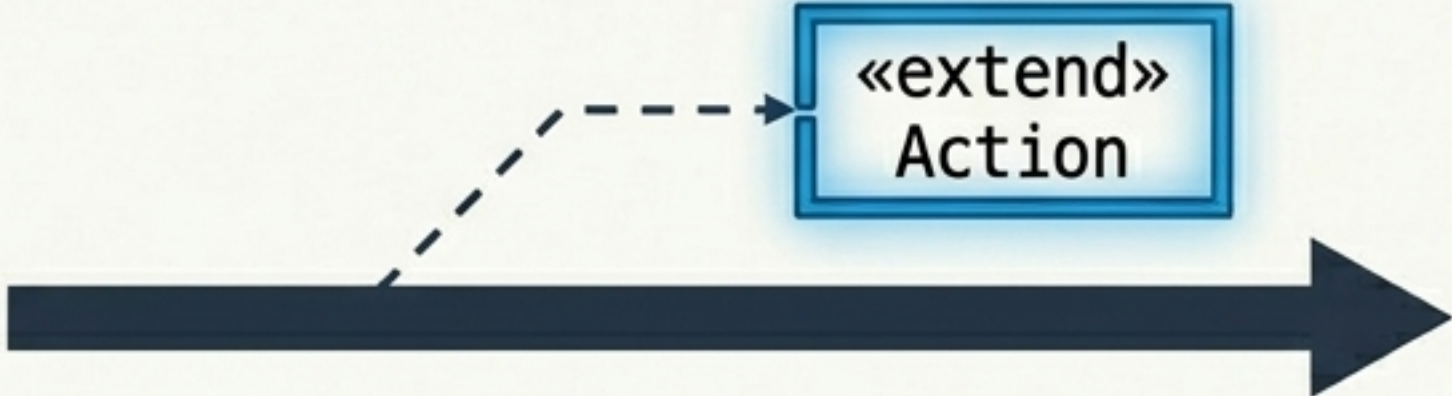
5. 基本事件流 (Main Flow): 填表 -> 系统校验 -> 创建记录 -> 发短信 -> 成功。

最典型的成功路径, 不要塞入
if-else 分支

6. 异常事件流 (Alt Flow): 邮箱错误(阻止) / 已报名(提示) / 短信失败(仅记录日志)。

基本流的失败分支全部走向这里

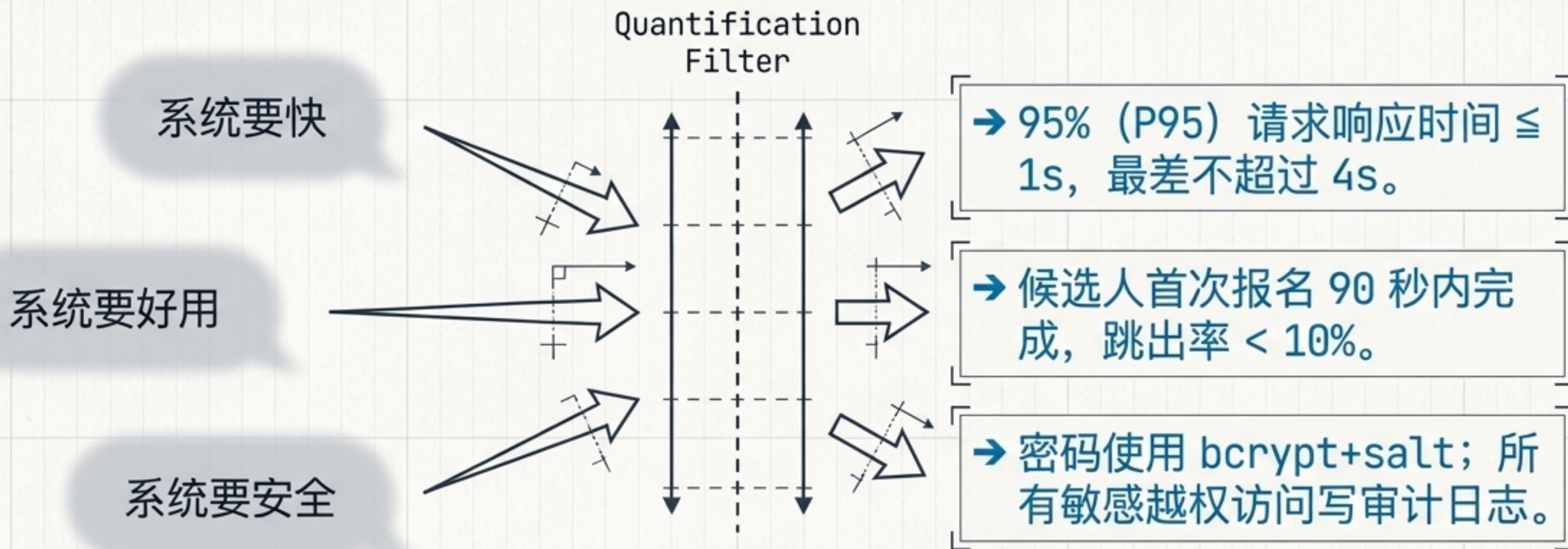
终结纠纷：«include» vs «extend»

«include» (包含)	«extend» (扩展)
 • 基础用例完整性：必须依赖才完整 • 触发时机：必然执行 • 工程化类比：函数调用必经分支 • 示例：提交报名 «include» 校验邮箱	 • 基础用例完整性：不依赖自身也完整 • 触发时机：条件性执行 • 工程化类比：AOP 钩子/插件机制 • 示例：提交报名 «extend» 找回密码

诊断法则： 如果去掉这个动作，主流程就跑不通了，那就是 include；
如果去掉后主流程依然成立，那就是 extend。

工具三：非功能需求 (NFR) — 决定产品能走多远

核心法则：不能测量的需求，不是真需求。没有数字 = 伪需求。



(*注：永远不要只看平均值，平均值会掩盖长尾性能问题*)

NFR 规范公式 = 描述 + 业务理由 + 量化验收标准

揭秘高可用性：「稳」的数学真相

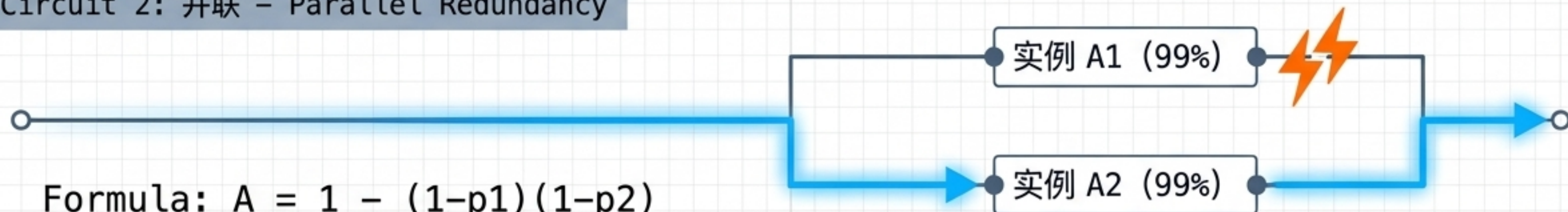
Circuit 1: 串联 - Serial Constraint



$$\text{Formula: } A = p1 \times p2$$

系统总体可用性 = **98.01%**. 任一环节挂 → 整体崩溃 (指标雪崩)。

Circuit 2: 并联 - Parallel Redundancy



$$\text{Formula: } A = 1 - (1-p1)(1-p2)$$

系统总体可用性 = **99.99%**. 任一环节存活 → 整体存活。

工程真理：冗余是提升可用性的唯一技术手段。主备、双活、多副本，本质全部是并联数学的工程实现。

最小需求骨架：三位一体的工程闭环

上下文图 (Context Diagram)

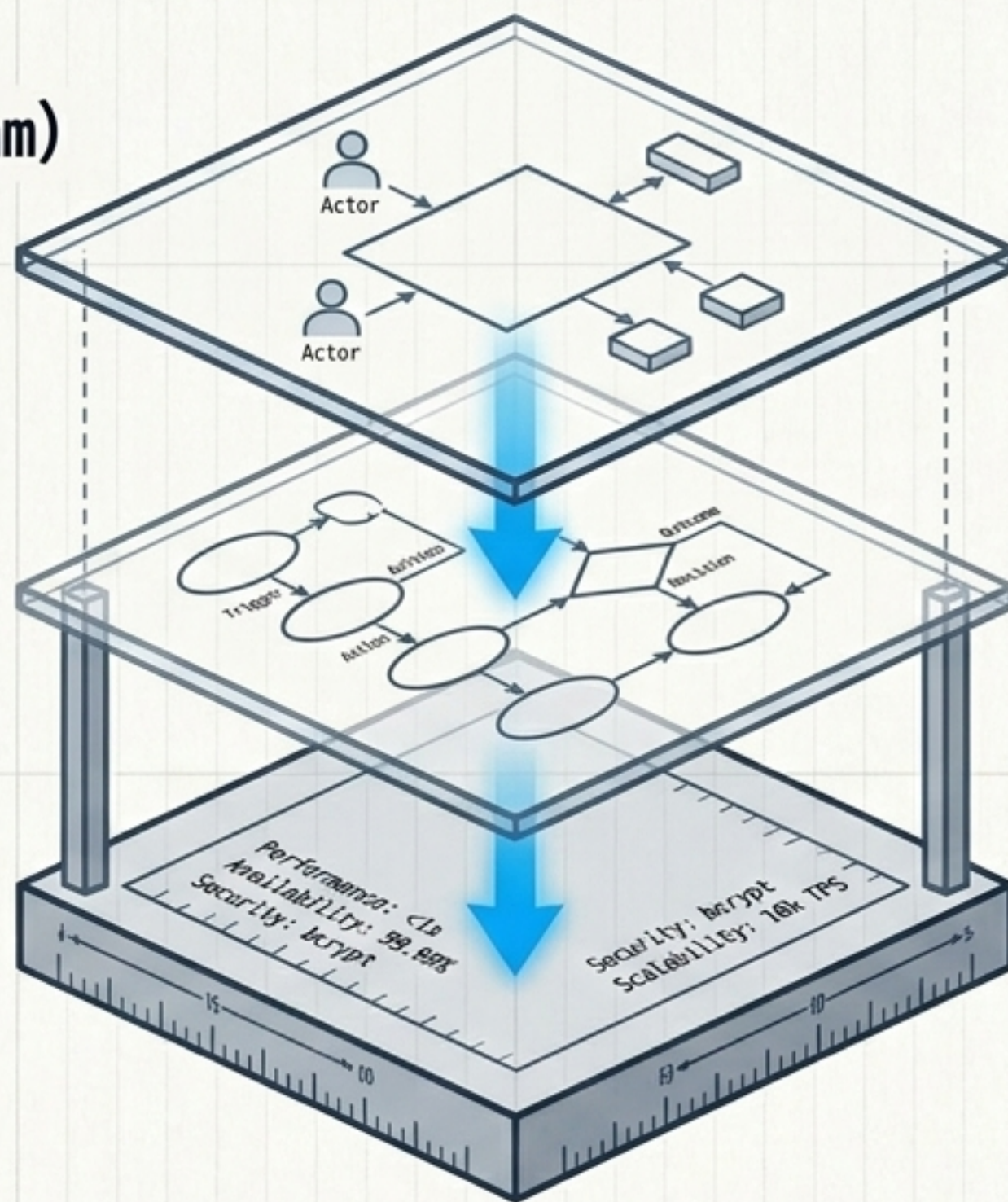
定义了系统的 Actor 与外部边界。

用例 (Use Cases)

精确描绘了边界内由 Actor 触发的行为流转。

非功能需求 (NFRs)

将执行流程牢牢约束在可测量的质量红线内。



Synthesis Insight

这不是三个独立的工具，而是一个层层递进的逻辑链。

跳过任何一环，都会导致最终产出的代码在现实中崩溃。

AI 时代的需求工程：重塑分工边界

 **AI 能力（加速器）**

 **人类职责（决策者）**

业务需求 - Business Layer

生成业务追问清单



确认真实的业务目标与数字

用例分析 - User Layer

基于上下文图批量扩展候选用例



核心审查与裁剪，剔除无业务价值节点

非功能需求 - Software Layer

提供行业通用的量化指标模板



结合公司现状拍板阈值与容错成本

第一法则 (The Prime Directive)

**AI 是最强的加速器，但判断权和责任永远留在人类这边。
绝不能让 AI 替你做业务决策。**

避坑清单与行动指南

最高 ROI 的 5 个原则

- ✓ 1. 一句话需求必须先归位三层（业务/用户/软件），再谈排期。
- ✓ 2. 先画上下文图——投资 5 分钟，挡住 30% 的下游遗漏。
- ✓ 3. 用例名坚持动宾格式，前后置条件必须是系统可检测状态。
- ✓ 4. 非功能需求拒绝形容词，全部数字化（性能看高百分位）。
- ✓ 5. 用 AI 提效，但人为判断负责。

课后挑战 (Next Steps)

[输入需求]:

做一个 Workshop 学员产出物提交与导师评分系统。

[要求产出]:

- 1 份业务需求（含可度量目标）
- 1 张上下文图
- 1 个标准用例描述（6 大核心字段）
- 3 条量化非功能需求